Khối	Vai trò
FIFO + giao tiếp ADC	Nhận mẫu ECG liên tục từ AFE/ADC qua SPI/I2C
Lọc số	Loại DC baseline, nhiễu 50/60 Hz, chạy trên RISC-V để dễ kiểm tra
Window generator / line buffer	Cấp cửa sổ K × Cin cho Conv1D mà không tạo ma trận im2col lớn
Scratchpad BRAM, weight buffer	Giữ tile input, weight, output gần systolic array
DMA + double buffer	Chồng thời gian đọc dữ liệu với tính toán
Accumulator INT32	Tích luỹ partial sum an toàn khi nhân INT8
Requantization	Scale INT32 về INT8 theo scale/zero-point của model
Activation, pooling, GAP	ReLU, max/average pooling, global average pooling rất rẻ phần cứng
MMIO register bank + interrupt	RISC-V ghi địa chỉ, kích thước tensor, khởi chạy accelerator và nhận hoàn thành
UART/debug counters	Xuất kết quả và đo cycle, latency, BRAM/DSP utilization

Model đầu tiên hỗ trợ:

Conv1D → ReLU → MaxPool

Conv1D → ReLU → Global Average Pool

Densse → Logits → Argmax

Phạm vi:

BatchNorm sẽ fuse vào Conv lúc compile.

Softmax chưa đụng tới nếu chỉ lấy logit lớn nhất.

Yêu cầu Compiler:

ONNX / TFLite

→ kiểm tra graph chỉ có các op được hỗ trợ

→ fuse Conv + BatchNorm + ReLU

→ quantize weights/activation INT8

→ tính shape và scale/zero-point từng tensor

→ chọn tile của systolic array

→ cấp phát scratchpad

→ sinh command descriptors + weights packed

→ xuất C header / binary model cho RISC-V

Outputs:

- weights.bin: weights INT8 đã pack.
- model_desc.h: shape, scale, địa chỉ buffer, thông số Conv.
- Danh sách lệnh: LOAD → CONV → REQUANT → POOL → STORE.
- Driver C chạy trên RISC-V, ghi các descriptor qua MMIO.

Tham khảo: https://github.com/ucb-bar/gemmini

Timeline

1. **Mô hình và golden reference**: Làm CNN 1D nhỏ trên Python, chia train/test theo bệnh nhân, không chia ngẫu nhiên từng beat. Lượng tử hóa INT8 và tạo bộ kiểm thử input/output.
2. **RTL**: Conv1D INT8 1 kênh, đối chiếu từng output với python fixed-point. Thêm ReLU, pooling và dense.
3. **Accelerator hoàn chỉnh chưa có compiler**: Nạp weight và thông số tĩnh. RISC-V chỉ ghi thanh ghi để chạy.
4. **SoC RISC-V + accelerator**: Dùng giao tiếp AXI/Wishbone/MMIO. Nếu không dùng được CPU tự xây hiện tại thì dùng có sẵn. (Tham khảo: https://github.com/YosysHQ/picorv32)
5. **Micro – compiler**: Parser ONNX giới hạn, sinh descripto và packed weight. Export model ra ONNX.
6. **Dòng ECG real-time**: Sau khi chạy đúng trên bộ tăng tốc, gắn AFE/ADC, FIFO, Preprocessing và Sliding Window.

Tài liệu

- RISC-V + scratchpad + DMA + Systolic Array: https://github.com/ucb-bar/gemmini
- FINN – Học compiler biến Quantized Neural Network thành FPGA dataflow architecture, dùng hiểu quantization, graph transform và packing weight: https://github.com/Xilinx/finn

- TVM VTA – Tham khảo thiết kế accelerator + compiler + runtime/driver hoàn chỉnh: https://github.com/apache/tvm-vta
- Hls4ml – Dùng làm nền cho CNN 1D trên FPGA trước khi viết RTL, hỗ trợ Conv1D và nhiều HLS backend: https://github.com/fastmachinelearning/hls4ml
- MIT-BIH Arrhythmia – Phân loại nhịp/beat, 48 bản ghi ECG hai kênh có annotation: https://physionet.org/content/mitdb/1.0.0/
- PTB-XL – Bài toán ECG 12-lead, 21.837 bản ghi 10 giây từ 18.885 bệnh nhân: https://physionet.org/content/ptb-xl/1.0.0/